

Задача 1: Современный проект на C++

Цель задачи

Подготовить проект на C++ согласно современным промышленным стандартам.

Навыки студента

- Самостоятельно подготовить проект на C++ по современным стандартам
- Уметь разбираться в ошибках сборки современного C++ проекта на платформах Linux и Windows
- Уметь разработать систему сборки проекта на C++ с использованием CMake
- Уметь писать и запускать юнит тесты на библиотеке GTest
- Уметь формировать отчёты о тестировании
- Уметь определять внешние зависимости и заниматься их менеджментом через менеджер пакетов Conan
- Уметь использовать средства автоматического соблюдения кодового формата и соглашений clang-tidy и clang-format
- Уметь создавать документацию встроенную в C++ код и собирать документацию для внешнего пользователя
- Уметь реализовывать установку приложения из исходных кодов

Требования к выполнению задачи

Проект должен:

- Иметь 1 статическую библиотеку
 - Функционал должен быть по минимуму, не заострять внимание на языке, а сделать просто склейку из библиотек чтобы работало
- Иметь 1 исполняемый файл, использующий статическую библиотеку и реализующий интерфейс командной строки для неё
 - Должно быть организовано логирование ошибок, библиотека логирования должна быть использована внешняя
- Иметь загрузку внешних зависимостей через Conan
- Иметь модульные тесты для библиотеки (в виде GTest исполняемого файла)
- Иметь сборочную систему на CMake
- Уметь запускать тесты и генерировать отчёты о их прохождении
- Уметь запускать форматирование кода согласно файлу clang-format
- Уметь запускать статический анализатор clang-tidy, с настройками из файла
- Уметь генерировать документацию библиотеки инструментом Doxygen
- Уметь устанавливать собранные бинарные артефакты из исходных кодов

Подготовительные материалы

- Формулировка задачи и требований к интерфейсу и функционалу исполняемого файла и библиотеки:
 - Требуется создать статическую библиотеку libcompressor
 - Содержит enum libcompressor_CompressionAlgorithm со значениями
 - libcompressor_Zlib
 - libcompressor_Bzip
 - Содержит определение структуры буфера struct libcompressor_Buffer с двумя членами данных
 - char* data

- int size
- Содержит функцию сжатия входного буфера в буфер
 - Сигнатура:
 - libcompressor_Buffer
 - libcompressor_compress(libcompressor_CompressionAlgorithm algo, libcompressor_Buffer input)
 - Поведение: Принимает алгоритм сжатия и реализует сжатие входного буфера данным алгоритмом с использованием библиотеки zlib или bzip, аллоцируя память под выходной буфер через функцию std::malloc() (начальный размер выходного буфера установить как размер входного буфера + 1024 байта), возвращает выходной буфер.
 - В случае ошибки возвращает буфер с data == NULL && size == 0
 - Параметры сжатия zlib Z_DEFAULT_COMPRESSION
 - Параметры сжатия bzip
 - Blocksize 1
 - Verbosity 0
 - Workfactor 0
- Созданные функции, перечисления и структуры должны быть размечены комментариями Doxygen
- Требуется написать набор юнит тестов для функции libcompressor_compress используя библиотеку Gtest, которые проверяют для двух алгоритмов сжатия что:
 - Если входной буфер не пустой то и выходной не пустой
 - Если входной буфер пустой или имеет нулевой размер, то выходной буфер пустой
 - Если на вход алгоритму libcompressor_Zlib подается буфер со строкой "test_string" (без учета в длине завершающего \0) то выходной буфер содержит


```
{0x78,0x9c,0x2b,0x49,0x2d,0x2e,0x89,0x2f,0x2e,0x29,0xca,0xcc,0x4b,0x07,0x00,0x1c,0x79,0x04,0xb7,}
```
 - Если на вход алгоритму libcompressor_Bzip подается буфер со строкой "test_string" (без учета в длине завершающего \0) то выходной буфер содержит


```
{0x42,0x5a,0x68,0x31,0x31,0x41,0x59,0x26,0x53,0x59,0x4a,0x7c,0x69,0x05,0x00,0x00,0x04,0x83,0x80,0x00,0x00,0x82,0xa1,0x1c,0x00,0x20,0x00,0x22,0x03,0x68,0x84,0x30,0x22,0x50,0xdf,0x04,0x99,0xe2,0xee,0x48,0xa7,0x0a,0x12,0x09,0x4f,0x8d,0x20,0xa0,}
```
 - При выполнении тесты должны генерировать отчет в формате xml по пути \${CMAKE_BINARY_DIR}/test_results.xml
- Требуется создать исполняемый файл compressor.exe который:
 - Принимает 2 аргумента строки, первый строка "zlib" или "bzip", второй произвольная строка символов
 - Сжимает строку из второго аргумента с использованием метода из первого аргумента и библиотеки libcompressor
 - Выводит выходной буфер в stdout по одному байту (формат "%0.2hhx"), после вывода буфера полностью добавляется перевод строки "\n"
 - Для вывода ошибок используется библиотека spdlog с уровнем логирования ERROR в stderr
 - Ошибкой считается:
 - Если аргументов меньше 2
 - Если первый аргумент не строка "zlib" или "bzip"
 - Если функция libcompressor_compress возвращает пустой буфер
 - Во всех случаях требуется запись в лог и выход с кодом EXIT_FAILURE
 - При удачном завершении работы выходной код должен быть EXIT_SUCCESS

- Формат проекта:
 - | .clang-format
 - | .clang-tidy
 - | .gitignore
 - | CMakeLists.txt
 - | conanfile.txt
 - +---compressor
 - | | CMakeLists.txt
 - | \---src
 - | compressor.cpp
 - \---libcompressor
 - | CMakeLists.txt
 - +---include
 - | \---libcompressor
 - | libcompressor.hpp
 - +---src
 - | libcompressor.cpp
 - \---tests
 - CMakeLists.txt
 - libcompressortest.cpp
- Библиотека и исполняемый файл должны быть устанавливаемые из исходных кодов (с использованием функционала CMake install() и модуля GNUInstallDirs)
- Библиотека и исполняемый файл должны собираться со следующими настройками:
 - Стандарт C++: C++20
 - Флаги предупреждения компилятора:
 - MSVC: /W4 /WX
 - GCC/Clang: -Wall -Wextra -Wpedantic -Werror
- В репозитории должен быть подготовленный pipeline GitHub Actions, который прогоняет этапы сборки проекта

Ссылки на материалы и справочные данные

- Реализация компрессии на bzip https://sourceware.org/pub/bzip2/docs/v101/manual_3.html#SEC36
- Реализация компрессии на zlib https://zlib.net/zlib_how.html
- Генерация документации Doxygen в CMake https://cmake.org/cmake/help/latest/module/FindDoxygen.html#command:doxygen_add_docs
 - Для корректной генерации документации установите в CMake:
 - set (DOXYGEN_OUTPUT_DIRECTORY \${CMAKE_BINARY_DIR}/docs)
 - set (DOXYGEN_EXTRACT_ALL YES)
- Скачать Doxygen https://github.com/doxygen/doxygen/releases/tag/Release_1_12_0
- Тесты с CTest https://cmake.org/cmake/help/latest/command/add_test.html и <https://cmake.org/cmake/help/latest/manual/ctest.1.html#run-tests>
- clang-tidy и clang-format распространяются совместно с проектом LLVM и компилятором clang. Скачать их можно на официальной странице релизов на GitHub <https://github.com/llvm/llvm-project/releases>
- Чтобы получить файл compile_commands.json (это называется “compilation database”, файл необходим для запуска clang-tidy и clang-format) необходимо:
 - В проекте после project() вызвать установку переменной set (CMAKE_EXPORT_COMPILE_COMMANDS YES)
 - На Linux файл появится в билд директории
 - На Windows, потребуется использовать генератор CMake отличный от “Visual Studio *”, так как он не поддерживает создание compile_commands.json
 - Для этого нужно установить систему сборки [Ninja](#)
 - [Создать профиль conan](#) с Ninja в качестве билд системы

- Собрать зависимости через `conan install` с новым профилем
- Далее, потребуется открыть папку проекта в среде разработки MSVC:
 - Открыть новый терминал `cmd.exe`
 - Перейти в директорию проекта
 - Вызвать скрипт: `call "C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Auxiliary\Build\vcvars64.bat"`. Он устанавливает переменные окружения MSVC
 - В том же терминале `cmd` вызывается строка запуска `cmake: cmake.exe -B build -G "Ninja"`
 - После того как отработает `cmake` в папке `build` появится `compile_commands.json`
- Для `clang-format` за основание берётся стиль Google
 - —
 - `BasedOnStyle: Google`
 - `IndentWidth: 2`
 - —
 - `Language: Cpp`
 - `DerivePointerAlignment: false`
 - `PointerAlignment: Left`
 - `ColumnLimit: 120`
 - ...
- Для `clang-tidy` за основание берётся файл сгенерированный через `clang-tidy --dump-config`